Meta Decks & Card Breakdown API Reference

Synchronized endpoint specification for fetching top tournament archetypes (MasterDuelMeta) and aggregated TCG decklists (YuGiOhMeta).

---

1. Architectural Overview & Synchronization Workflow

Both `masterduelmeta.com` and `yugiohmeta.com` expose RESTful JSON endpoints under `/api/v1/deck-types` powered by a MongoDB/FeathersJS query syntax. When used in tandem inside the WikiYugi-oh backend service, they enable automated discovery of current competitive archetypes and their optimal TCG card ratios:

1. **Step 1 — Fetch Tiered Archetypes (MasterDuelMeta):** Query the first endpoint to retrieve an ordered array of all active competitive deck types with a tournament power greater than or equal to `1`, sorted from highest meta share to lowest.
2. **Step 2 — Fetch Aggregated TCG Decklist (YuGiOhMeta):** Extract the `name` string from any deck returned in Step 1 (e.g., `"Dracotail"`) and pass it into the second endpoint's `name` query parameter with `aggregate=aboveThresh` to obtain the statistical Main, Extra, and Side Deck card breakdown.
3. **Step 3 — Hydrate Card Entities (YGOPRODeck API):** Map the card names returned in Step 2 against your local card cache or YGOPRODeck API to instantiate the project's OOP domain classes (`Mazo`, `CartaMonstruo`, `CartaMagicaTrampa`).

2. Endpoint 1: Most Popular Tournament Decks

Retrieves a lightweight list of ranked tournament archetypes ordered by current competitive representation.

```
GET https://www.masterduelmeta.com/api/v1/deck-types?
tournamentPower[$gte]=1&limit=0&sort=-
tournamentPower&fields=name,tournamentPower,tournamentPowerTrend
```

2.1 Query Parameters Breakdown

Parameter	Value	Description & Purpose
`tournamentPower[$gte]`	`1`	MongoDB comparison operator (`$gte` = greater than or equal to). Filters out rogue or inactive decks with a tournament power score below `1.0`.

Parameter	Value	Description & Purpose
`limit`	`0`	Disables pagination limits and returns all matching competitive deck archetypes in a single response array.
`sort`	`-tournamentPower`	The leading minus sign (`-`) sorts results in descending order so the #1 most dominant tournament deck appears at index `0` .
`fields`	`name,tournamentPower,tournamentPowerTrend`	Projection filter that strips heavy nested objects from the payload, returning only the deck ID, archetype name, power rating, and trend indicator.

2.2 Response Payload Schema (JSON Array)

The endpoint returns a top-level JSON array of deck-type objects. Each object contains the projected fields plus the default MongoDB document identifier `_id` :

```
[
  {
    "_id": "66a1f9c2e4b0a12d8c93e101",
    "name": "Dracotail",
    "tournamentPower": 14.8,
    "tournamentPowerTrend": 1.25
  },
  {
    "_id": "65f4b210e4b0c84a1209b412",
    "name": "Snake-Eye Fiendsmith",
    "tournamentPower": 11.4,
    "tournamentPowerTrend": -0.65
  }
]
```

2.3 Response Field Definitions

Field Path	Data Type	Description
`[]._id`	`String`	Unique 24-character MongoDB ObjectId representing the deck archetype document on MasterDuelMeta.
`[].name`	`String`	Official archetype display name (e.g., `"Dracotail"`). **Foreign Key:** Pass this exact string (URL-encoded) into Endpoint 2's `name` query parameter.

Field Path	Data Type	Description
`[].tournamentPower`	`Number (Float)`	Weighted representation score calculated from recent tournament tops. Higher values indicate higher tier status (Tier 1, Tier 2, Tier 3).
`[].tournamentPowerTrend`	`Number (Float | null)`	Delta score comparing current tournament power against the previous period. Positive values indicate rising popularity; negative values indicate declining share.

3. Endpoint 2: TCG Deck Card Breakdown

Retrieves the aggregated TCG card usage statistics and standard copy ratios for a specific archetype name.

```
GET https://www.yugiohmeta.com/api/v1/deck-types?name=Dracotail&limit=1&aggregate=aboveThresh
```

3.1 Query Parameters Breakdown

Parameter	Example Value	Description & Purpose
`name`	`Dracotail`	Exact case-sensitive match for the archetype name obtained from Endpoint 1. Always apply `encodeURIComponent(name)` when building URLs dynamically.
`limit`	`1`	Restricts the query to return at most 1 matching archetype document inside the response array.
`aggregate`	`aboveThresh`	Server-side aggregation flag that computes the `deckBreakdown` object and filters out obscure one-off tech cards that fall below the inclusion threshold.

3.2 Response Payload Schema (JSON Array)

Returns a single-element JSON array containing the archetype metadata and the computed `deckBreakdown` object, which lists every staple card along with how frequently it is run at 1, 2, or 3 copies in Main/Extra vs. Side Deck:

```
[
  {
    "_id": "66b8d410e4b09a3c1e742099",
    "name": "Dracotail",
    "tcgPower": 12.5,
    "deckBreakdown": {
      "totalDecks": 48,
      "cards": [
        {
          "card": {
            "_id": "60e1c8a2e4b0123456789abc",
            "name": "Ash Blossom & Joyous Spring"
          },
          "count": 46,
          "atOne": 0,
          "atTwo": 2,
          "atThree": 44,
          "isSide": false
        },
        {
          "card": {
            "_id": "61a2b3c4e4b0987654321def",
            "name": "Nibiru, the Primal Being"
          },
          "count": 38,
          "atOne": 3,
          "atTwo": 10,
          "atThree": 25,
          "isSide": true
        }
      ],
      "engines": []
    }
  }
]
```


3.3 Response Field Definitions

Field Path	Data Type	Description & Deck-Building Logic
`[0]._id`	`String`	Unique identifier of the archetype in the YuGiOhMeta TCG database.
`[0].name`	`String`	Archetype name matching the query parameter (e.g., `"Dracotail"`).
`[0].deckBreakdown.totalDecks`	`Number (Int)`	Total number of tournament-topping TCG decklists analyzed to build this aggregation sample.
`[0].deckBreakdown.cards[]`	`Array<Object>`	List of cards meeting the popularity threshold across the sampled tournament decks.

Field Path	Data Type	Description & Deck-Building Logic
`...cards[].card._id`	`String`	Internal YuGiOhMeta MongoDB ID for the card.
`...cards[].card.name`	`String`	Exact English TCG card name. Use this string to query your `GET /api/cartas/:name` or YGOPRODeck API endpoint.
`...cards[].count`	`Number (Int)`	Number of sampled decks (out of `totalDecks`) that included this card. Inclusion rate = `(count / totalDecks) * 100`.
`...cards[].atOne`	`Number (Int)`	Number of decks running exactly **1 copy** of this card.
`...cards[].atTwo`	`Number (Int)`	Number of decks running exactly **2 copies** of this card.
`...cards[].atThree`	`Number (Int)`	Number of decks running exactly **3 copies** of this card.
`...cards[].isSide`	`Boolean`	`false` if the card statistics apply to the Main/Extra Deck; `true` if the statistics represent Side Deck usage.

4. Algorithm for Reconstructing a Playable TCG Deck

Because Endpoint 2 returns aggregated statistical distributions (`atOne`, `atTwo`, `atThree`) rather than a single static 40-card list, your backend API should compute the optimal copy count for each card using the following deterministic rules:

- **Filter out Side Deck entries (or separate them):** Entries with `isSide: false` populate the Main and Extra Decks, while entries with `isSide: true` populate the Side Deck array.
- **Determine Optimal Copy Quantity (Mode Calculation):** Compare `atThree`, `atTwo`, and `atOne` for each card entry. Assign the copy count (`3`, `2`, or `1`) that has the highest value: `const copies = (atThree >= atTwo && atThree >= atOne) ? 3 : (atTwo >= atOne ? 2 : 1);`
- **Sort by Inclusion Rate:** Sort cards in descending order by `count / totalDecks` so core engine cards (90%–100% usage) are added to the `Mazo` instance first before filling remaining slots with popular staples.
- **Cross-Format Fallback:** Occasionally, a Master Duel archetype name may not yet exist in the TCG or may have `totalDecks: 0` on YuGiOhMeta. If Endpoint 2 returns an empty array `[]` or an empty `cards` array, fall back to querying `https://www.masterduelmeta.com/api/v1/deck-types?name=<DeckName>&limit=1&aggregate=aboveThresh`, which uses the exact same JSON schema.

> **Backend Proxy Requirement (CORS & Caching):** Calling `masterduelmeta.com` and `yugiohmeta.com` directly from browser-side Vanilla JS will trigger Cross-Origin Resource Sharing (CORS) blocks. Always fetch these two endpoints inside your Node.js/Express backend server and expose them to your frontend via your own route (e.g., `GET /api/mazos/meta` and `GET /api/mazos/meta/:name`).

5. Backend API Implementation Reference (Node.js / Express)

The service module below synchronizes both endpoints, calculates optimal card quantities, and formats the response for your frontend `Mazo` and `Carta` classes:

```
// services/metaDecksService.js
const MDM_URL = 'https://www.masterduelmeta.com/api/v1/deck-types';
const YGM_URL = 'https://www.yugiohmeta.com/api/v1/deck-types';

// 1. Get Top Tournament Decks from MasterDuelMeta
async function getTopTournamentDecks() {
  const query = '?tournamentPower[$gte]=1&limit=0&sort=-
tournamentPower&fields=name,tournamentPower,tournamentPowerTrend';
  const response = await fetch(`${MDM_URL}${query}`);
  if (!response.ok) throw new Error(`MasterDuelMeta API error:
${response.status}`);
  return await response.json(); // Returns [{ _id, name, tournamentPower,
tournamentPowerTrend }, ...]
}

// 2. Get Aggregated TCG Cards for a Specific Deck Name from YuGiOhMeta
async function getTcgDeckCardsByName(deckName) {
  const encodedName = encodeURIComponent(deckName);
  const query = `?name=${encodedName}&limit=1&aggregate=aboveThresh`;
  const response = await fetch(`${YGM_URL}${query}`);
  if (!response.ok) throw new Error(`YuGiOhMeta API error: ${response.status}`);

  const data = await response.json();
  if (!Array.isArray(data) || data.length === 0 || !data[0].deckBreakdown) {
    return { deckName, totalDecks: 0, mainAndExtraCards: [], sideCards: [] };
  }

  const { totalDecks, cards = [] } = data[0].deckBreakdown;

  // Map statistical breakdown into concrete card quantities
  const normalizedCards = cards.map((entry) => {
    const { atOne = 0, atTwo = 0, atThree = 0, count = 0, isSide = false } =
entry;
    const optimalCopies = (atThree >= atTwo && atThree >= atOne) ? 3 : (atTwo >=
atOne ? 2 : 1);
    const usagePercentage = totalDecks > 0 ? Number(((count / totalDecks) *
100).toFixed(1)) : 0;

    return {
      cardIdMeta: entry.card?._id || null,
      name: entry.card?.name || 'Unknown Card',
      quantity: optimalCopies,
      usagePercentage,
      distribution: { atOne, atTwo, atThree },
      isSide: Boolean(isSide)
    };
  }).sort((a, b) => b.usagePercentage - a.usagePercentage);

  return {
    deckId: data[0]._id,
    deckName: data[0].name,
    totalDecksAnalyzed: totalDecks,
    mainAndExtraCards: normalizedCards.filter((c) => !c.isSide),
    sideCards: normalizedCards.filter((c) => c.isSide)
  };
}
```

6. Recommended WikiYugi-oh Internal API Endpoints

To integrate this workflow cleanly into the REST API contract established for your project, expose these two routes on your server:

Method	Internal Route	Upstream Source	Response to Frontend
`GET`	`/api/mazos/meta`	MasterDuelMeta Endpoint 1	`200 OK` — Array of popular tournament decks with `name`, `tournamentPower`, and `tournamentPowerTrend`.
`GET`	`/api/mazos/meta/:name`	YuGiOhMeta Endpoint 2 + YGOPRODeck API	`200 OK` — Complete deck breakdown object containing `mainAndExtraCards` and `sideCards` with quantities and card details.